Untrusted Prover, Trusted Referee: 28 Lean Proofs for `formal-conjectures` from an AI Agent Judged by an External Checker

Daniel Puri
github.com/danielpuri1901/erdos-lean-formalization

28 October 2026

Abstract

Google DeepMind's `formal-conjectures` library states hundreds of research problems in Lean 4, most with a hole (`sorry`) where a proof belongs. On 3 and 4 October 2026 I had AI agents (Claude Opus 5.5 and Claude Fable 5.1, through Claude Code) fill 28 such holes: 26 results that were already settled on paper, among them two disproofs that had no machine-checked proof, and two Graffiti.pc conjectures that other people had proved in the months before. Each proof was judged by a checker the agent cannot change: a separate repository rebuilds the agent's commit, replaces the agent's copy of the statement with a frozen one taken with `type_of%` from the library, and runs Comparator in a sandbox. The same agents spent one hour each on 16 problems the library marks open and ten more attempts on two of them. They solved none. On two problems they proved new special cases in Lean (the Catch-Up conjecture for $N \leq 16$; the OEIS A000224 conjecture for n even, prime, a prime power, or $p^e q$). The whole project used 244 USD of model credit. A Lean proof of a known result cost about 2 USD and took the agent 7 to 35 minutes; the open problems took more than 150 USD and stopped, every time, at the step that needs a new idea.

1 What was done

The library `formal-conjectures`[1] writes research problems as Lean statements. A statement tagged *research solved* with a `sorry` is a known theorem without a formal proof. A statement tagged *research open* is, as far as the library knows, an open problem.

I ran two kinds of task.

1. **Known results.** An AI agent gets one statement, the published argument in a few lines, and a working rule set. It writes a Lean proof of the exact statement and submits it.

2. **Open problems.** An AI agent gets one open statement and one hour: understand the claim, try to break it by computation, try to prove it, report. The two most promising problems then got a longer run with a manager agent and ten attempts.

Everything the agent writes is untrusted. Success has one definition: a green status written by a separate checker repository on a pushed commit.

2 The checker

The agent works in its own repository and can edit anything there. When it is done it pushes a commit whose message starts with `submit(<target>):`. A second repository, `erdos-lean-checker`, watches for such commits and runs the following steps in GitHub Actions.

[1] `https://github.com/google-deepmind/formal-conjectures`, pinned at commit `df3f12d7bd06`.

1. Check out the agent's commit. Delete its build directory, its Lake configuration and its `Challenge.lean`.

2. Copy in the frozen `Challenge.lean` for the target. The frozen file states the theorem as `theorem x : type_of% @Lib.name` (or its negation), so the statement is the library's by construction. A separate test compares the two elaborated types and fails the run on any difference.

3. Build the frozen challenge with the pinned Mathlib and library.

4. Run Comparator[2] on the agent's `Solution` module inside a landrun sandbox. Comparator rebuilds the solution, checks that the proved theorem has the frozen type, exports and re-checks the proof term with `lean4export`, and rejects any axiom other than `propext`, `Quot.sound` and `Classical.choice`. `native_decide`, `implemented_by` and `unsafe` are rejected with the rest.

5. Write the commit status `comparator/<target>`: success only if Comparator's exit code is zero and its last line is `Your solution is okay!`.

Before the real runs the checker got one correct proof and seven cheating ones: a `sorry`, an added axiom, `native_decide`, a redefined constant, a proof of a weaker statement, a changed challenge, and a solution that printed the success line during its own build. Six were rejected at once. The seventh, the printed success line, passed because the first version of the checker searched its log for that line. The verdict is now the exit code, and the fake proof is a permanent test.

3 Method for the agent

Each known result ran as one plain Claude Code cloud session (Claude Opus 5.5, highest effort setting) on its own branch. The branch holds the frozen statement, a `Solution.lean` with `sorry`, a lessons file shared across sessions, and a task file with the goal, the known mathematics in a few lines, the working rules and the submit rule. The working rules follow Terence Tao's published recipe for Claude Code and Lean: write the argument in prose first, build a skeleton with every lemma stated and `sorry`, make it compile, then fill the holes one at a time; commit after each step; keep every kernel check small because the checker machine has 7.8 GB of memory. Time boxes were 15 to 90 minutes of work. The agent polled the commit status itself and fixed and resubmitted on failure. Of the 28 accepted proofs, 26 passed on their first submission; two day-one runs on Green 72 needed a second submission after the checker's machine ran out of memory.

4 Results on known results

Table 1 lists the 28 accepted proofs. Each entry links from `PROOFS.md` in the repository to the proof files at the accepted commit and to the checker's log. Proofs with large finite checks (the Witt design, the two TxGraffiti disproofs, $R(4,4)$, $R(3,5)$, the cube digraph) use `Nat` bitmasks, a Boolean checker with a soundness lemma, and `decide +kernel` in chunks; this pattern was learned in the first such run and reused through the lessons file.

[2] `https://github.com/leanprover/comparator`, commit `3927ad3`.

Table 1: The 28 proofs accepted by the checker. "Negated" means the proof is of the negation of the library statement.

Result and library statement	Note
Erdős problem 105, $n-4$ variant `Erdos105.erdos_105.variants.sub_four` (negated)	The variant with $\lvert A\rvert = \lvert B\rvert+4$ is false (counterexample found by forum user Core_65536 in July 2026 with an AI-assisted search, who also checked it in Lean).
Green's problem 72, $N \le 60$ `Green72.no_three_in_line_le`	No three in a line on an $N \times N$ grid: the maximum is $2N$ for $3 \le N \le 60$, from the published point sets of Flammenkamp and Prellberg.
Clark and Suen (2000) `VizingConjecture.vizing_conjecture.variants.clark_suen`	$\gamma(G)\gamma(H) \le 2\gamma(G \,\square\, H)$.
Suen and Tarr (2012) `VizingConjecture.vizing_conjecture.variants.suen_tarr`	$\gamma(G)\gamma(H) + \min(\gamma(G), \gamma(H)) \le 2\gamma(G \,\square\, H)$.
TxGraffiti conjecture 3 is false `Arxiv.2507.17780.tx_graffiti_conjecture_3` (negated)	$i(G) \le \mu^*(G)$ fails for a 3-regular graph on 48 vertices (graph from ipitchford/txgraffiti-order48-successor).
Green's problem 66, trivial bound `Green66.green_66.variants.trivial_bound`	For large X some sum of two squares lies in $[X - CX^{1/4}, X]$.
Danilov (1982) `Hall.danilov`	Hall's conjecture fails with any exponent above $1/2$.
Gauss's circle bound `GaussCircleProblem.error_le`	$\lvert E(r)\rvert \le 2\sqrt{2}\,\pi r$ for large r.
TxGraffiti conjecture 2 is false `Arxiv.2507.17780.tx_graffiti_conjecture_2` (negated)	$Z(G) \le \alpha(G) + 1$ fails for Fischer's 24-vertex graph (arXiv:2607.23664).
Witt design $S(5, 8, 24)$ `SteinerSystems.steiner_system_5_8_24`	The 759 octads of the Golay code form a Steiner system.
Giuga (1950) `AgohGiuga.isStrongGiuga_iff`	Strong Giuga numbers are the Carmichael numbers with $\sum_{p\mid n} 1/p - 1/n \in \mathbb{N}$.
$R(4, 4) = 18$ `RamseyNumbers.ramsey_number_four_four`	
Non-abelian Leinster group `LeinsterGroup.exists_nonabelian_leinster_group`	$S_3 \times C_5$ is a Leinster group.
Cube digraph, $m = 2$ `ClaudesCycles.cube_hamiltonian_arc_decomposition_impossible_m2`	No Hamiltonian arc decomposition of the digraph on $(\mathbb{Z}/2)^3$ (Aubert and Schneider, 1982).
$R(3, 5) = 14$ `RamseyNumbers.ramsey_number_three_five`	
Dihedral Leinster groups `LeinsterGroup.dihedral_is_leinster_iff_odd_perfect`	D_n is Leinster iff n is an odd perfect number.
Giuga numbers have 9 prime factors `AgohGiuga.agoh_giuga.variants.le_primeFactors_card_of_isStrongGiuga`	
Weak Giuga numbers `AgohGiuga.isWeakGiuga_iff_sum_primeFactors`	Characterisation by $\sum_{p\mid n} 1/p - 1/n \in \mathbb{N}$.

Result and library statement	Note
2-increasing triples, squares `Arxiv.1609.08688.maximalLength_ge_of_isSquare`	$F(n) \geq n^{3/2}$ for square n (Gowers and Long).
Exact circle count `GaussCircleProblem.exact_form_floor`	$N(r) = 1 + 4\sum_i(\lfloor r^2/(4i+1)\rfloor - \lfloor r^2/(4i+3)\rfloor)$; Jacobi's two-squares count proved on the way.
$R(3,4) = 9$ `RamseyNumbers.ramsey_number_three_four`	
Integer complexity of 3^n `Mathoverflow75792.complexity_three_pow`	$\|3^n\| = 3n$ (Selfridge).
OEIS A112970 `OeisA112970.conjecture1_value`	$a(2^n) = \lfloor n^2/4\rfloor + 1$.
$R(3,3) = 6$ `RamseyNumbers.ramsey_number_three_three`	
Strong Giuga implies Carmichael `AgohGiuga.agoh_giuga.variants.isStrongGiuga_implies_isCarmichael`	
2-increasing triples, powers `Arxiv.1609.08688.maximalLength_pow`	$F(n) = n^e$ for one $n > 1$ gives $F(m) \geq m^e$ for infinitely many m.
Graffiti.pc conjecture 19 `WrittenOnTheWallII.GraphConjecture19.conjecture19`	Proved by Chen, Wang and Feng in July 2026; the library still tags it open. Found in the open-problem sweep.
Graffiti.pc conjecture 100 `WrittenOnTheWallII.GraphConjecture100.conjecture100`	Proved by Kias Henry in August 2026; the library links his proof. Found in the open-problem sweep.

Not all of these are first formal proofs. When this paper was written the library already linked proofs by other people for the Erdős 105 variant, for A112970, for the Giuga-to-Carmichael implication and for Graffiti.pc 100, and a theorem named `ramsey_number_four_four` exists in Bhavik Mehta's Ramsey project, so $R(4,4) = 18$ very likely has an earlier Lean proof with other definitions. The two TxGraffiti disproofs are the only ones for which I found no earlier Lean proof; the counterexample graphs are from Fischer and from the `txgraffiti-order48-successor` repository, and the Lean proofs add the machine check.

Two known results were not finished in their time box: Chen and Saito's theorem that minimum degree 3 forces a cycle of length divisible by 3, and the Danzer–Klee bound 2^{n-1} for snakes in the n-cube. Both tasks came without a proof sketch.

5 Results on open problems

Sixteen statements tagged open got one hour each (83 USD in all). Two sessions returned full Lean proofs that the checker accepted: Graffiti.pc conjectures 19 and 100. Both conjectures had been proved by other people in July and August 2026; the library's tag was out of date, and my candidate scan had checked only the tag. A search for earlier work then showed that five of the sixteen were already settled: three proved and two disproved (TxGraffiti 2 and 3, which became known-result tasks above).

Two problems with real progress and no known solution got a longer run (68 USD): a manager session (Claude Fable 5.1) kept a notebook, gave single steps to fresh Opus workers with a 30-minute box, had a second worker attack every paper argument, and stopped after ten attempts.

- **Catch-Up** (two players take numbers from $1..N$; the one behind keeps taking; the claim is a draw under perfect play when $N(N+1)/2$ is even). Proved in Lean for every $N \leq 16$ through strategy certificates checked by the kernel, and reduced to four statements about unions of pairs $\{a, a+1\}$ that are checked by computer inside $\{1..20\}$ and not proved.
- **OEIS A000224** ($n^2 \equiv 1 \pmod{a(n)(a(n)-1)}$ iff n is an odd prime, with $a(n)$ the number of squares modulo n). Proved in Lean for n even, prime, a prime power, or $p^e q$ with p, q prime and any e. A counterexample would need at least two distinct prime factors each with exponent at least 2, or at least three distinct prime factors; none exists below 1.6×10^{10}.

These partial theorems build from a clean checkout with no `sorry`, but the checker did not judge them, because it only judges a full frozen statement, and no mathematician has reviewed them. The pattern on every open problem was the same: the agent did the routine work fast (small cases, searches, reductions, Lean proofs of all of that) and stopped at one step that needs a new idea; more attempts gave more cases around that step and not the idea.

6 Costs

Step	USD	Outcome
Checker, cloud setup, Erdős 105, six runs on Green 72	27	2 proofs, harness versus bare comparison
Clark–Suen and Suen–Tarr	5	2 proofs
Sixteen open problems, one hour each	83	2 proofs of already-solved conjectures, 14 partial
Two problems, ten attempts each	68	partial results, no solution
One more A000224 attempt, TxGraffiti 3 disproof	17	one more family of cases, 1 disproof
Twenty-three known results	44	21 proofs, 2 partial
Total	244	

On Green's problem 72 a harness with a manager agent and fresh workers was compared with a bare session, three runs each. All six passed; the bare session was faster and cheaper (32 minutes and 2 USD on average against 70 minutes and 4.7 USD). The known results were too short to show a benefit from the harness, and the open problems did not yield to it.

7 What went wrong

The checker ran out of memory and disk on GitHub's small runner (fixed by removing unused SDKs and sizing a swap file from free disk), its polling loop crashed on an empty API reply (fixed with a test), and on the second evening GitHub stopped starting jobs for private repositories on my account; the checker repository is public since then, which its trust model does not depend on. My own mistake was to trust the library's *open* tag: two "solved open problems" were not open.

8 Limitations

No open problem was solved. The proofs of known results fill holes in one library and are not claimed as first proofs except where stated. The partial results on the two open problems have not been reviewed by a mathematician. The statement checks that tie each frozen challenge to the library were written by the AI; I checked them with small examples and playable versions of the problems, not by reading the Lean. The prior-work search used the web and a GitHub code search and may have missed earlier proofs.

9 Reproducibility

The repository holds every accepted proof under `proofs/<id>/` with the frozen statement, the proof files, the data and the session's notes, and `PROOFS.md` links each one to the accepted commit (a complete Lake project with the pinned manifest) and to the checker's log. The checker, its frozen statements, its negative tests and its workflow are in `danielpuri1901/erdos-lean-checker`. The run records, costs and session identifiers are in `docs/`.

AI declaration

The Lean proofs were written by Claude Opus 5.5 in Claude Code cloud sessions, with task files written by Claude Fable 5.1 under my direction; the stage-2 manager was Claude Fable 5.1. The checker, the candidate scans, the prior-work search and this text were written by Claude Fable 5.1 and Claude Opus 5.5 under my direction and with my decisions at every stage. I am not a mathematician. The mathematics in the known results is other people's: the $n-4$ counterexample for Erdős 105 is Core_65536's (the $n-3$ counterexamples are Xichuan's), the point sets for Green 72 are Flammenkamp's and Prellberg's, the 24-vertex graph is Fischer's, the 48-vertex graph is from `ipitchford/txgraffiti-order48-successor`, and the theorems carry their authors' names in Table 1. The problems come from `formal-conjectures` (Google DeepMind), the checking tool is Comparator (the Lean team) and the sandbox is landrun.